



INTRODUCTION TO JAVA PROGRAMMING

CCE 109/L- CCE 102/L

JAVA QUICKSTART

- In Java, every application begins with a class name, and that class must match the filename.
- Let's create our first Java file, called `MyClass.java`, which can be done in any text editor (like Notepad).

SAMPLE PROGRAM

```
public class NewClassSample {  
    public static void main(String[] args){  
        System.out.println("Hello World!");  
        System.out.println("Another Sample");  
    }  
}
```

SAMPLE PROGRAM

- Every line of code that runs in Java must be inside a **class**.
- In our example, we named the class **NewClassSample**.
- A class should always start with an uppercase first letter.
- **Note:** Java is case-sensitive: "MyClass" and "myclass" has different meaning.
- The name of the java file **must match** the class name. When saving the file, save it using the class name and add ".java" to the end of the filename. To run the example above on your computer, make sure that Java is properly installed.

SAMPLE PROGRAM OUTPUT

- **Hello World!**
- **Another Sample**

THE `main` METHOD

- The `main()` method is required, and you will see it in every Java program:

```
public static void main(String[] args)
```

Any code inside the `main()` method will be executed.

The `main()` method is the entry point into the application.

You don't have to understand the keywords before and after `main`.

IMPORTANT POINTS TO REMEMBER:

- every Java program has a **class** name which must match the filename, and that
- every program must contain the **main()** method.

`System.out.println()`

- Inside the `main()` method, we can use the `println()` method to print a line of text to the screen:

```
public static void main(String[] args) {  
    System.out.println("Hello World"); }  
}
```

IMPORTANT POINTS TO REMEMBER:

- The curly braces `{ }` marks the beginning and the end of a block of code.
- Each code statement must end with a semicolon.

JAVA COMMENTS

- Comments can be used to explain Java code, and to make it more readable. It can also be used to prevent execution when testing alternative code.
- **Single-line comments** start with two forward slashes (//).
- Any text between // and the end of the line is ignored by Java (will not be executed).

JAVA COMMENTS

- This example uses a single-line comment before a line of code:

```
// This is a comment  
System.out.println("Hello World");
```

This example uses a single-line comment at the end of a line of code:

```
System.out.println("Hello World"); // This is a  
comment
```

JAVA MULTI-LINE COMMENTS

- Multi-line comments start with `/*` and ends with `*/`.
- Any text between `/*` and `*/` will be ignored by Java.

This example uses a multi-line comment (a comment block) to explain the code:

```
/* The code below will print the words Hello World  
to the screen, and it is amazing */  
System.out.println("Hello World");
```

SINGLE OR MULTI-LINE COMMENTS?

- It is up to you which you want to use.
- Normally, `//` is used for short comments, and `/* */` for longer.

JAVA VARIABLES

- **Variables** are containers for storing data values.

In Java, there are different **types** of variables, for example:

- **String** – stores text, such as "Hello". String values are surrounded by double quotes
- **int** – stores integers (whole numbers), without decimals, such as 123 or -123
- **float** – stores floating point numbers, with decimals, such as 19.99 or -19.99
- **char** – stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
- **boolean** – stores values with two states: true or false

VARIABLE DECLARATION SYNTAX

- `Variable_type variable_name = value;`
- Where *variable_type* is one of Java's types (such as `int` or `String`), and *variable_name* is the name of the variable (such as `x` or `name`).
- The **equal sign** is used to assign values to the variable.

EXAMPLE:

- Create a variable called **subject** of type String and assign it the value "CCE 109/L".

ANSWER:

```
String name = "John";  
System.out.println(name);
```

EXAMPLE:

- To create a variable that should store a number, look at the following example:

Create a variable called **number** of type `int` and assign it the value **109**:

```
int number = 109;  
System.out.println(number);
```

JAVA VARIABLES

- You can also declare a variable without assigning the value, and assign the value later:

```
int number;  
number = 109;  
System.out.println(number);
```

JAVA VARIABLES

- Note that if you assign a new value to an existing variable, it will overwrite the previous value:

Change the value of number from 109 to 120:

```
int number = 109;  
number = 120; // number is now 120  
System.out.println(number);
```

FINAL VARIABLES

- You can add the final keyword if you don't want others (or yourself) to overwrite existing values (this will declare the variable as "final" or "constant", which means unchangeable and read-only):

Example:

```
final int myNum = 15;
```

```
myNum = 20; // will generate an error: cannot  
assign a value to a final variable
```

A DEMONSTRATION OF HOW TO DECLARE VARIABLES OF OTHER TYPES:

```
int myNum = 5;  
float myFloatNum = 5.99f;  
char myLetter = 'D';  
boolean myBool = true;  
String myText = "Hello";
```

WHAT DO YOU THINK IS THE OUTPUT OF THIS PROGRAM?

```
String name = "John";  
System.out.println(number);
```

WHAT DO YOU THINK IS THE OUTPUT OF THIS PROGRAM?

```
int number=15;  
number= 20;  
number=5;  
System.out.println(number);
```

WHAT DO YOU THINK IS THE OUTPUT OF THIS PROGRAM?

```
int number = 20;  
number = 30;  
System.out.println(mynumber);
```

DISPLAY VARIABLES

- The `println()` method is often used to display variables.
- To combine both text and a variable, use the `+` character:

Example:

- `String name = "John";`
- `System.out.println("Hello " + name);`

DISPLAY VARIABLES

- You can also use the + character to add a variable to another variable:

Example

```
String firstName = "John ";  
String lastName = "Doe";  
String fullName = firstName + lastName;  
System.out.println(fullName);
```

DISPLAY VARIABLES

- For numeric values, the + character works as a mathematical operator (notice that we use int (integer) variables here):

```
int x = 5; int y = 6;
```

```
System.out.println(x + y); // Print the value of x + y
```

- From the example above, you can expect:
- x stores the value 5
- y stores the value 6

Then we use the `println()` method to display the value of `x + y`, which is **11**

DECLARE MANY VARIABLES

- To declare more than one variable of the **same type**, use a comma-separated list:

```
int x = 5, y = 6, z = 50;  
System.out.println(x + y + z);
```

JAVA IDENTIFIERS

- All Java **variables** must be **identified** with **unique names**.
- These unique names are called **identifiers**.
- Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).
- **Note:** It is recommended to use descriptive names in order to create understandable and maintainable code:

JAVA IDENTIFIERS

- `// Good int minutesPerHour = 60;`
- `// OK, but not so easy to understand what m
actually is`
- `int m = 60;`

THE GENERAL RULES FOR CONSTRUCTING NAMES FOR VARIABLES (UNIQUE IDENTIFIERS) ARE:

- Names can contain letters, digits, underscores, and dollar signs
- Names must begin with a letter
- Names should start with a lowercase letter and it cannot contain whitespace
- Names can also begin with \$ and _ (but we will not use it in this tutorial)
- Names are case sensitive ("myVar" and "myvar" are different variables)
- Reserved words (like Java keywords, such as int or boolean) cannot be used as names

SAMPLE ACTIVITY:

- Create a variable named **carName** and assign the value Volvo to it.

ACTIVITY:

- Create a simple Java Program that displays the following information:

Sample:

My name is Iris Mae Mendoza.

I am a first year BS Computer Science student.